Rule Engine Design

AI-Powered Security Analytics Platform

Field	Value
Document ID	RULE-ENGINE-001
Version	1.0
Date	2026-07-18
Status	Draft
Language	Node.js + TypeScript
Architecture Reference	[ADR-001](file:///d:/AI%20SIEM/docs/architecture/decisions/ADR-001-modular-monolith.md)
Backend Detection	[BACKEND-002](file:///d:/AI%20SIEM/docs/backend-detection.md)
Database Design	[DB-001](file:///d:/AI%20SIEM/docs/database.md)

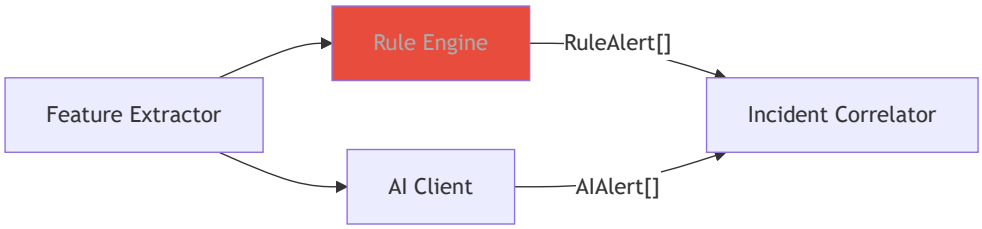
Table of Contents

1. Architecture
2. Execution Flow
3. Rule Storage
4. Rule Evaluation
5. Rule Priorities
6. Rule CRUD
7. Performance Optimization

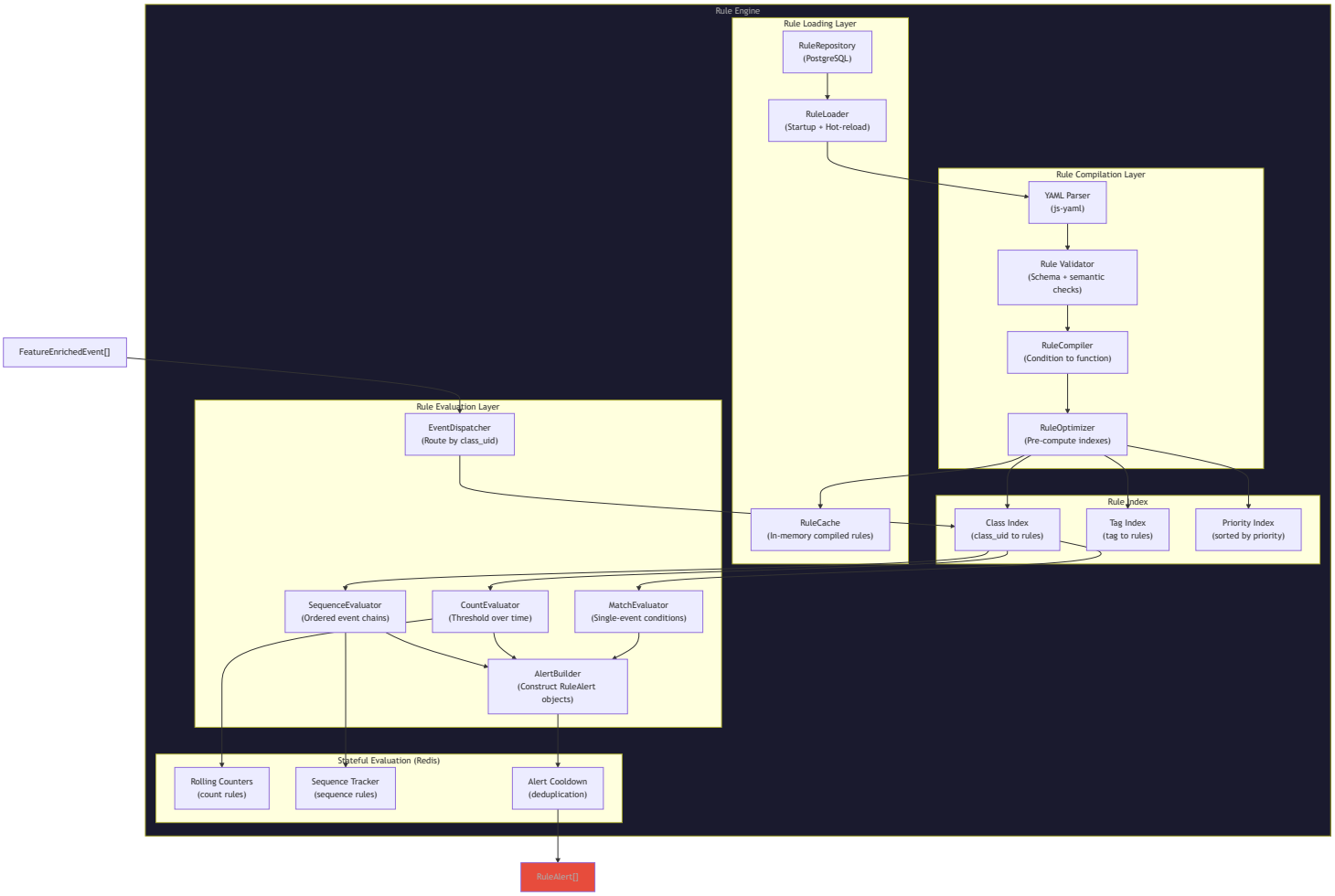
1. Architecture

1.1 Position in the Pipeline

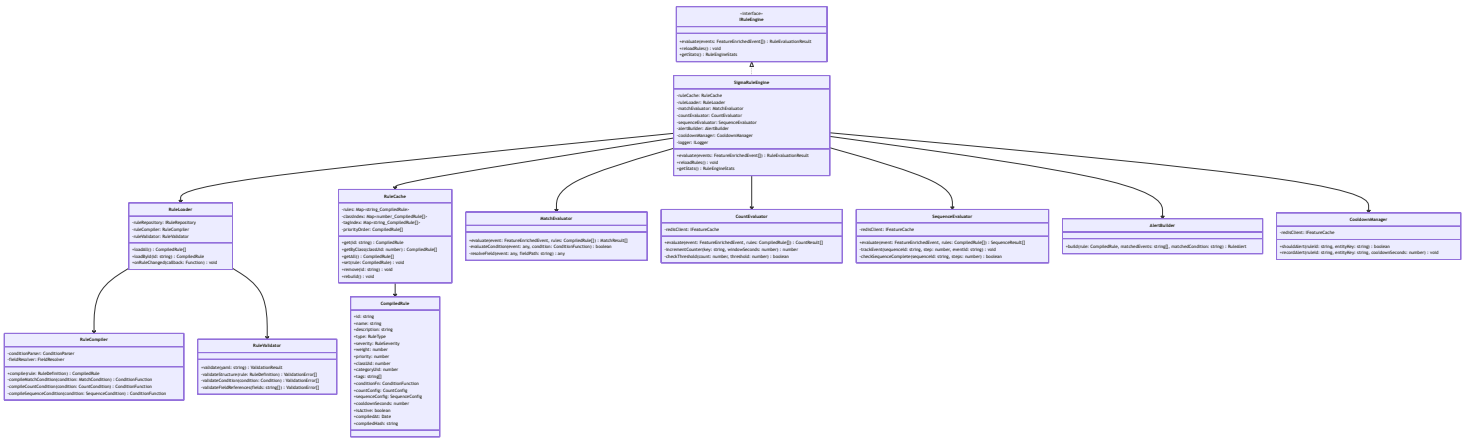
The Rule Engine sits in the **Detection Layer** of the backend pipeline. It receives feature-enriched events from the Feature Extractor and produces rule-based alerts that feed into the Incident Correlator.



1.2 Internal Architecture



1.3 Class Diagram



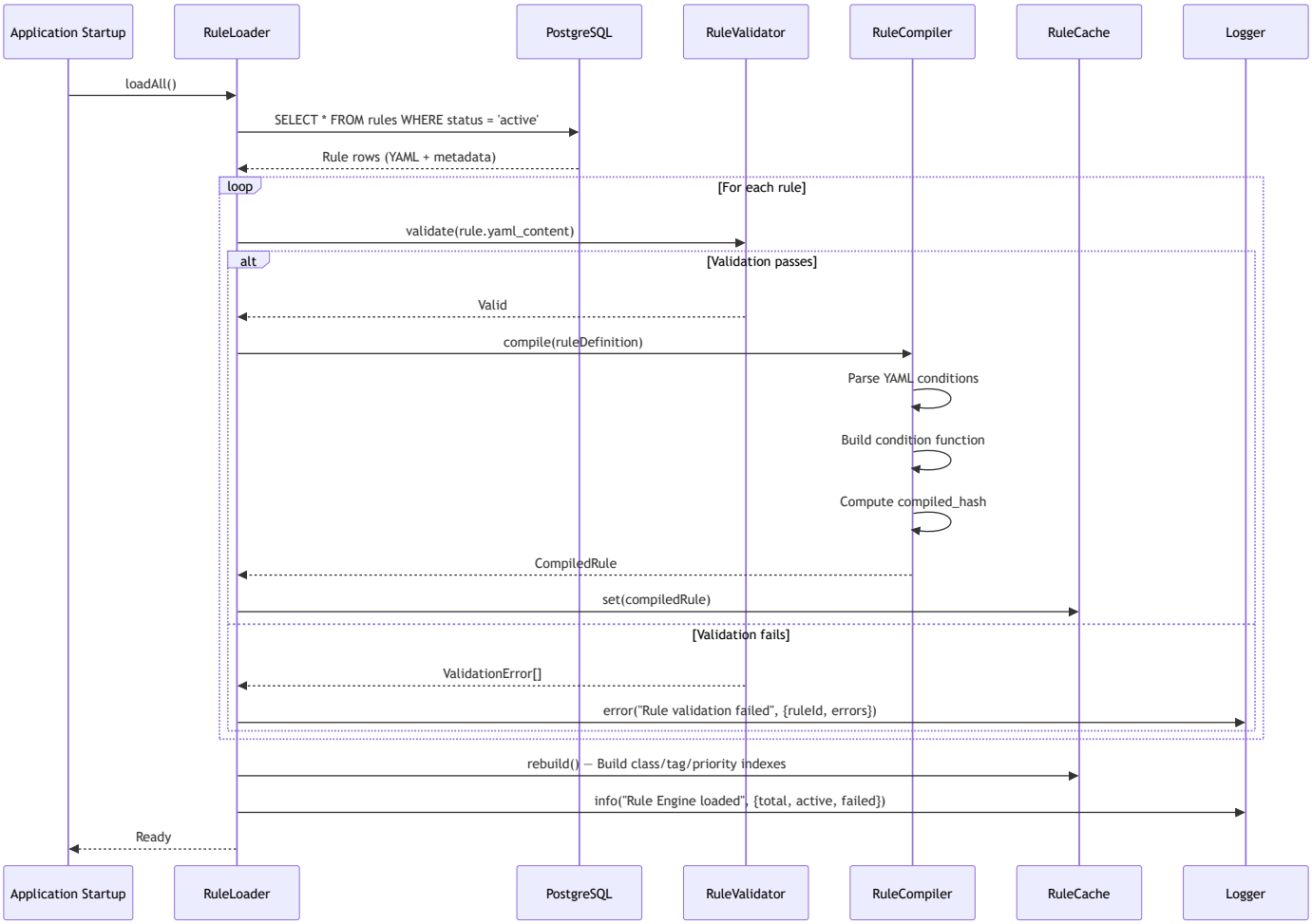
1.4 Design Principles

Principle	Application
Hot-reloadable	Rules are loaded from PostgreSQL into memory. CRUD operations trigger a targeted cache update — no restart needed
Compiled rules	Sigma YAML is compiled into JavaScript condition functions at load time. Evaluation is a function call, not a YAML re-parse

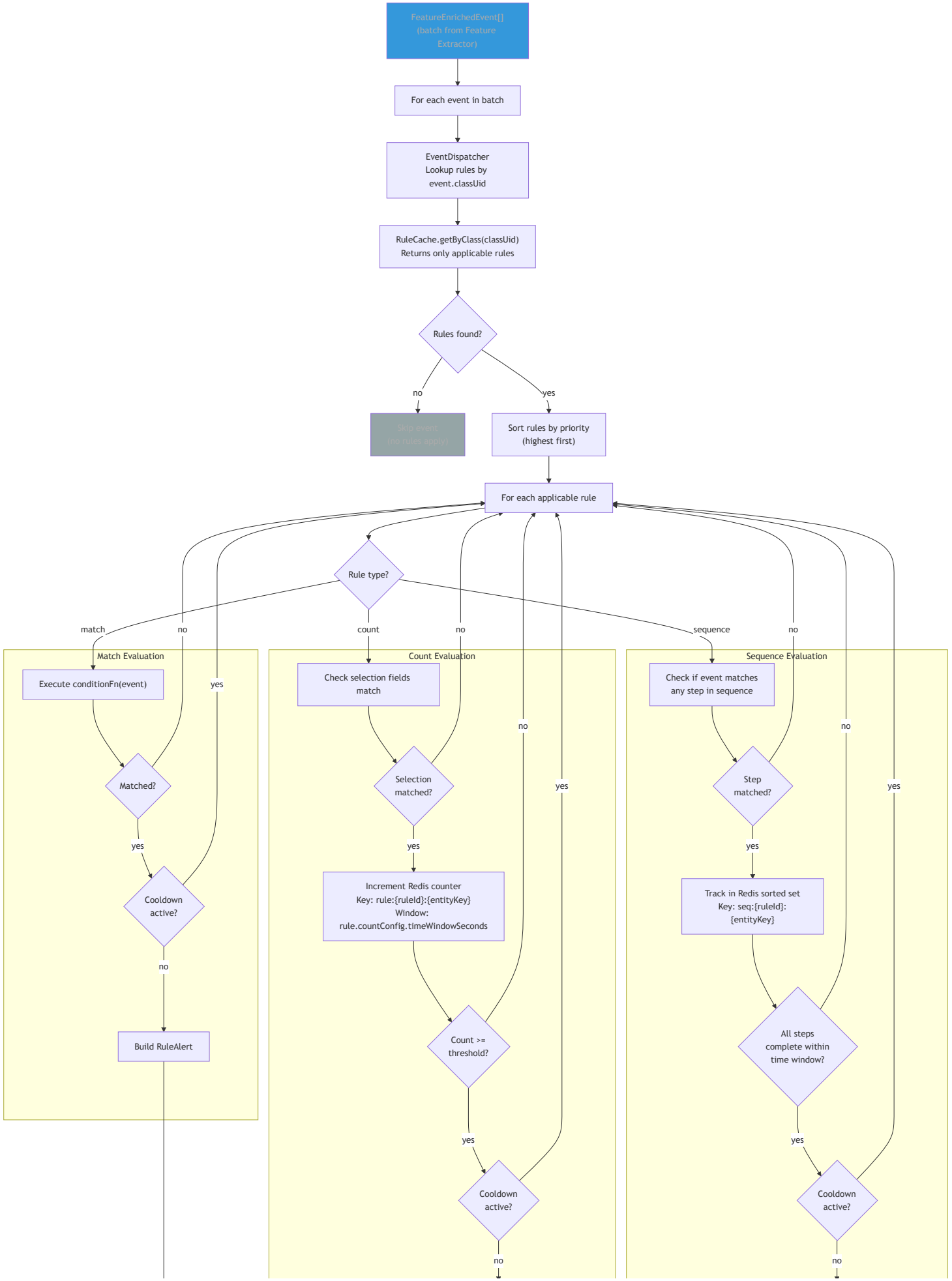
Principle	Application
Class-indexed dispatch	Events are routed only to rules that match their OCSF `class_uid`. Rules irrelevant to an event class are never evaluated
Stateful evaluation	Count and Sequence rules maintain state in Redis with TTL. State automatically expires — no cleanup needed
Cooldown deduplication	After a rule fires, it enters a cooldown period for that entity. Prevents 1000 identical alerts from a brute force burst

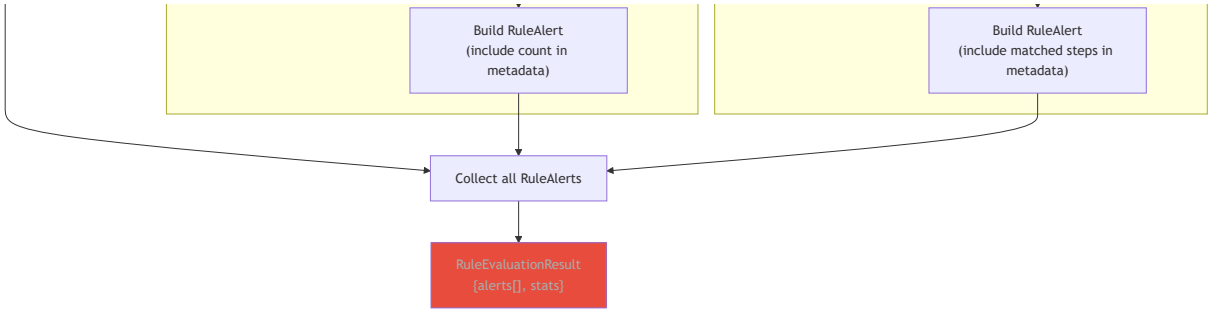
2. Execution Flow

2.1 Startup Flow



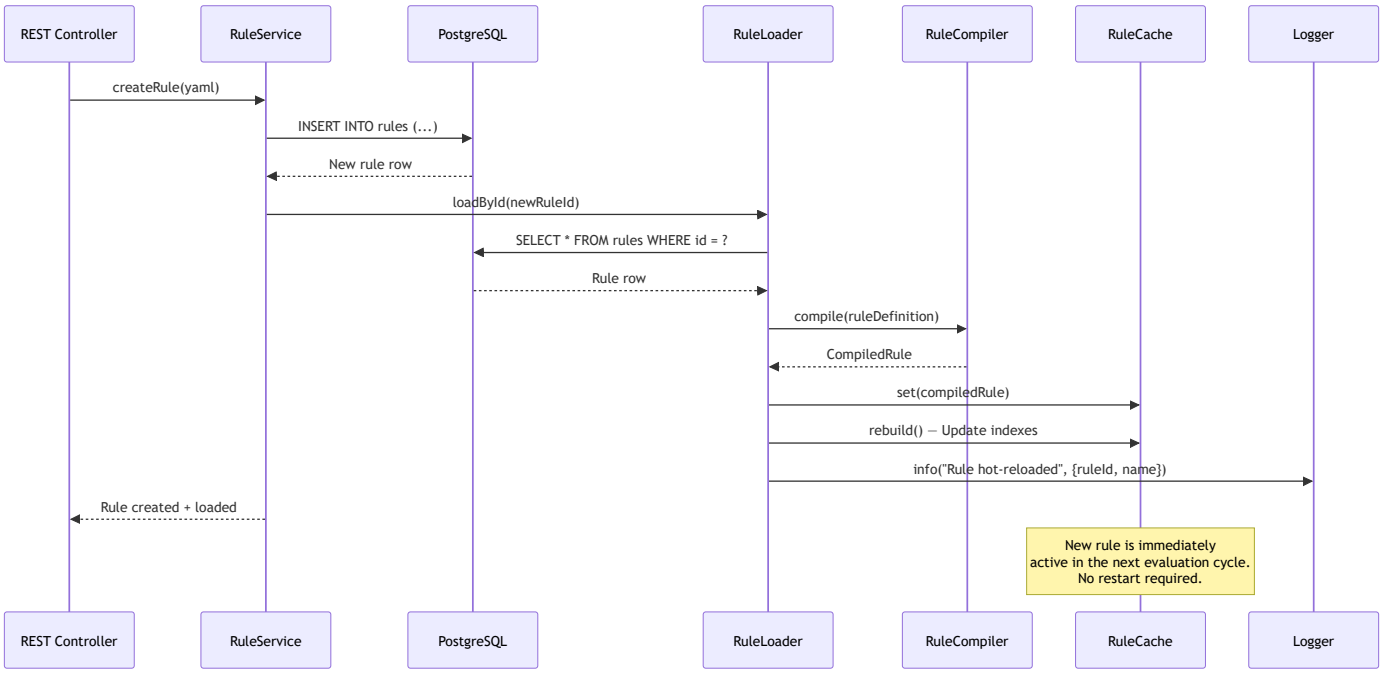
2.2 Event Evaluation Flow


FeatureEnrichedEvent[]
(batch from Feature Extractor)
For each event in batch
EventDispatcher
Lookup rules by event.classUid
RuleCache.getByClass(classUid)
Returns only applicable rules
Rules found?
no
yes
Skip event
(no rules apply)
Sort rules by priority
(highest first)
For each applicable rule
Rule type?
match
count
sequence
no
Match Evaluation
Execute conditionFn(event)
Matched?
yes
Cooldown active?
yes
no
Build RuleAlert
Count Evaluation
Check selection fields match
Selection matched?
Increment Redis counter
Key: rule:{ruleId}:{entityKey}
Window: rule.countConfig.timeWindowSeconds
Count >= threshold?
Cooldown active?
Sequence Evaluation
Check if event matches any step in sequence
Step matched?
Track in Redis sorted set
Key: seq:{ruleId}:{entityKey}
All steps complete within time window?
Cooldown active?



2.3 Hot-Reload Flow

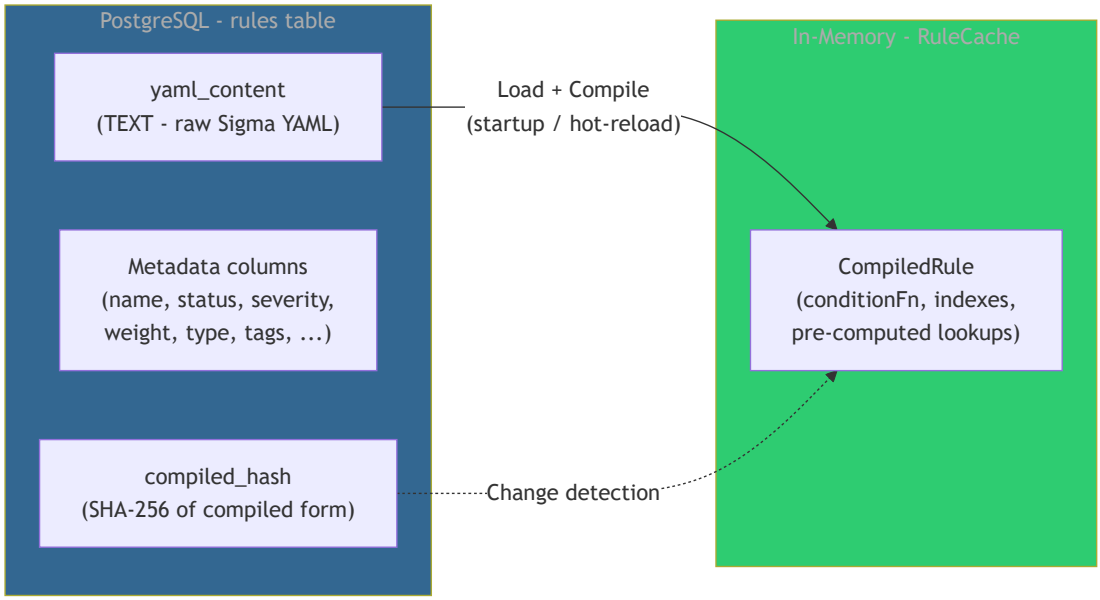
When a rule is created, updated, deleted, enabled, or disabled via the REST API, the Rule Engine is updated in real-time without restart.



3. Rule Storage

3.1 Rule Storage Model

Rules are stored in PostgreSQL in the `rules` table (defined in [DB-001](file:///d:/AI%20SIEM/docs/database.md)). The YAML content is the **source of truth**. Compiled representations are in-memory only.



3.2 Sigma Rule Schema

Every rule follows a standardized Sigma-inspired YAML schema:

```yaml
# ============================================================
# Sigma-Compatible Rule Schema
# ============================================================

# --- Identity ---
title: "Human-readable rule name"          # REQUIRED
id: "rule-unique-id-001"                   # REQUIRED (UUID or slug)
description: |                             # OPTIONAL
  Multi-line description of what this
  rule detects and why it matters.

# --- Metadata ---
status: active                             # REQUIRED: active | disabled | archived
level: high                                # REQUIRED: critical | high | medium | low | informational
weight: 0.8                                # REQUIRED: 0.0 - 1.0 (used in risk scoring)
priority: 100                              # OPTIONAL: 1-1000 (higher = evaluated first)
version: 1                                 # AUTO-INCREMENTED on update

# --- OCSF Source Filter ---
logsource:
  class_uid: 3002                          # REQUIRED: OCSF event class
  category_uid: 3                          # OPTIONAL: OCSF category

# --- Detection Logic ---
detection:
  # Selection: field conditions that must match
  selection:
    field_name: "exact value"              # Exact match
    field_name|contains: "substring"       # Substring match
    field_name|startswith: "prefix"        # Prefix match
    field_name|endswith: "suffix"          # Suffix match
    field_name|re: "regex_pattern"         # Regex match
    field_name|gt: 100                     # Greater than
    field_name|gte: 100                    # Greater or equal
    field_name|lt: 50                      # Less than
    field_name|lte: 50                     # Less or equal
    field_name|in:                         # In list
      - "value1"
      - "value2"
    field_name|exists: true                # Field exists
    field_name|not: "excluded_value"       # Not equal

  # Optional: additional selection groups (AND/OR logic)
  selection_extra:
    another_field: "value"

  # Filter: events to exclude from matching (NOT)
  filter:
    field_name: "legitimate_value"

  # Condition: how selections combine
  condition: "selection AND selection_extra AND NOT filter"

  # For count rules: threshold configuration
  count:
    field: "src_endpoint.ip"               # Group-by field for counting
    threshold: 10                          # Alert when count >= threshold
    timewindow: "5m"                       # Rolling time window (s/m/h)

  # For sequence rules: ordered step definitions
  sequence:
    timewindow: "10m"                      # All steps within this window
```

```
    by: "actor.user.name"                 # Entity that links the steps
    steps:
      - selection_step1:
          message|contains: "Failed password"
      - selection_step2:
          message|contains: "Accepted password"
      - selection_step3:
          class_uid: 3004                 # Authorization event

# --- Alert Configuration ---
alert:
  cooldown: "5m"                          # Suppress duplicate alerts for this entity+rule

# --- Classification ---
tags:
  - attack.credential_access             # MITRE ATT&CK tactic
  - attack.t1110                          # MITRE ATT&CK technique
  - compliance.pci_dss

falsepositives:
  - "Automated backup processes"
  - "Scheduled password rotation scripts"

references:
  - "https://attack.mitre.org/techniques/T1110/"
  - "https://schema.ocsf.io/1.1.0/classes/authentication"
```

3.3 Rule Field Resolution

Rule conditions reference event fields using **dot-notation paths**. The Rule Engine resolves these paths against `FeatureEnrichedEvent` objects.

Dot-Notation Path	Resolved From	Example Value
`class_uid`	`event.classUid`	`3002`
`severity_id`	`event.severityId`	`3`
`message`	`event.message`	`"Failed password for root"`
`src_endpoint.ip`	`event.srcEndpoint.ip`	`"192.168.1.100"`
`dst_endpoint.ip`	`event.dstEndpoint.ip`	`"10.0.0.5"`
`dst_endpoint.port`	`event.dstEndpoint.port`	`22`
`actor.user.name`	`event.actor.user.name`	`"root"`
`actor.process.name`	`event.actor.process.name`	`"sshd"`
`device.hostname`	`event.device.hostname`	`"web-server-01"`
`metadata.product.name`	`event.metadata.product.name`	`"sshd"`
`features.frequency.events_per_minute_src_ip`	`event.features.frequency.events_per_minute_src_ip`	`45`
`features.temporal.is_business_hours`	`event.features.temporal.is_business_hours`	`false`
`features.authentication.failed_login_count_10min`	`event.features.authentication.failed_login_count_10min`	`12`
`enrichments.geo_src.country`	`event.enrichments.geo_src.country`	`"CN"`

Note

Rules can reference **features** and **enrichments** directly. This means rules like "alert when `features.frequency.events_per_minute_src_ip > 30` AND `features.temporal.is_business_hours = false`" are fully supported. This bridges rule-based and statistical detection.

3.4 Condition Operators

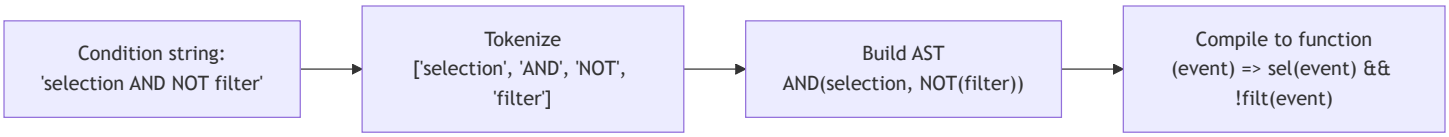
Operator	YAML Suffix	JavaScript Compiled Form	Example
Exact match	*(none)*	`value === target`	`message: "audit log cleared"`
Contains	`|contains`	`value.includes(target)`	`message|contains: "Failed password"`
Starts with	`|startswith`	`value.startsWith(target)`	`actor.user.name|startswith: "admin"`
Ends with	`|endswith`	`value.endsWith(target)`	`src_endpoint.ip|endswith: ".100"`
Regex	`|re`	`new RegExp(target).test(value)`	`message|re: "Failed.*root"`
Greater than	`|gt`	`value > target`	`severity_id|gt: 3`
Greater or equal	`|gte`	`value >= target`	`features.frequency.events_per_minute_src_ip|gte: 30`
Less than	`|lt`	`value < target`	`features.temporal.hour_of_day|lt: 6`
Less or equal	`|lte`	`value <= target`	`dst_endpoint.port|lte: 1024`
In list	`|in`	`targets.includes(value)`	`actor.process.name|in: ["mimikatz", "nc"]`
Not equal	`|not`	`value !== target`	`actor.user.name|not: "SYSTEM"`
Exists	`|exists`	`value !== undefined && value !== null`	`enrichments.geo_src|exists: true`

3.5 Condition Logic

Selections combine using boolean logic in the `condition` string:

Logic	Syntax	Meaning
AND	`selection AND selection_extra`	Both must match
OR	`selection OR selection_extra`	Either must match
NOT	`NOT filter`	Exclude events matching filter
Combined	`(selection OR selection_alt) AND NOT filter`	Grouped boolean logic

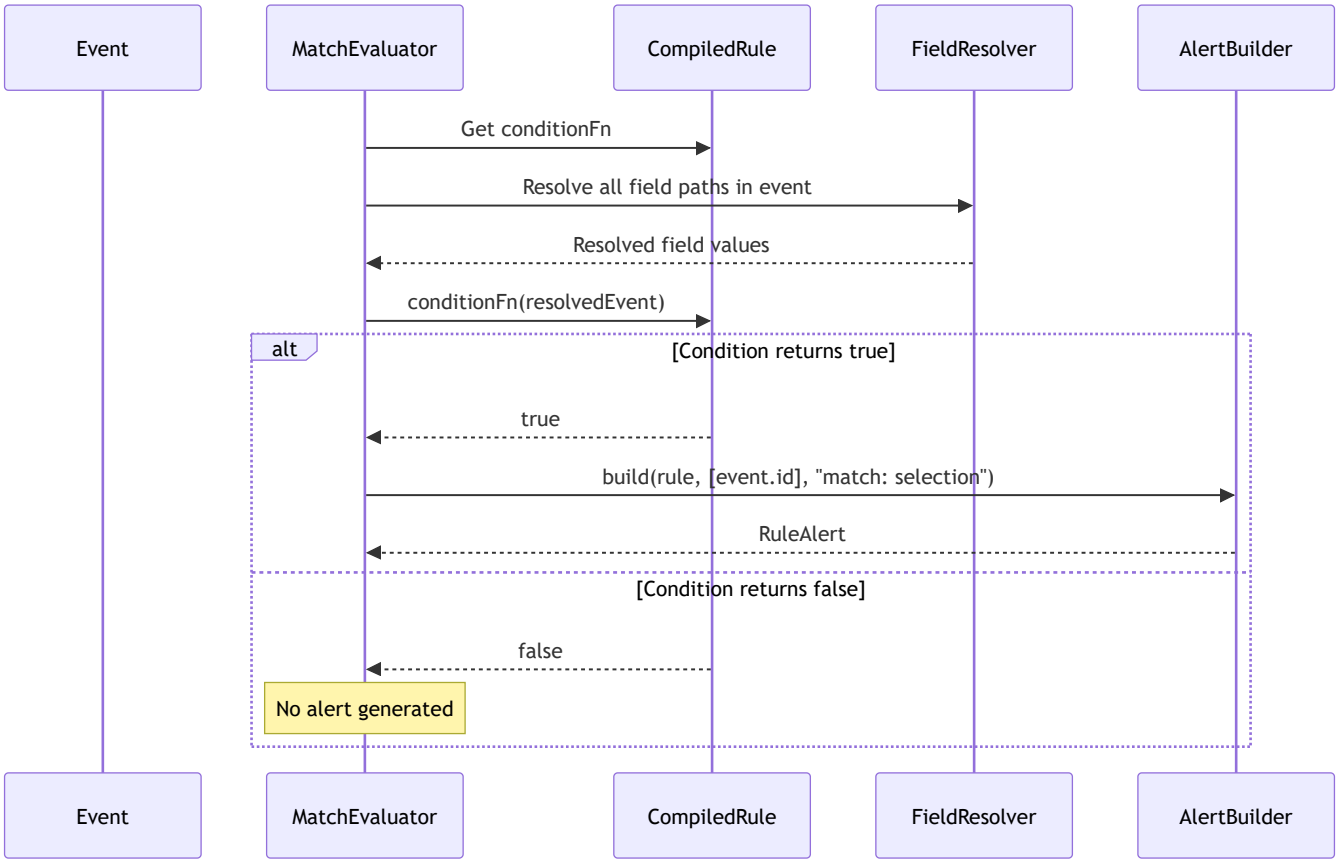
The RuleCompiler parses the condition string into an AST and compiles it to a nested JavaScript function:



4. Rule Evaluation

4.1 Match Rule Evaluation

Match rules evaluate a single event against a condition function. If the function returns `true`, the event matches.

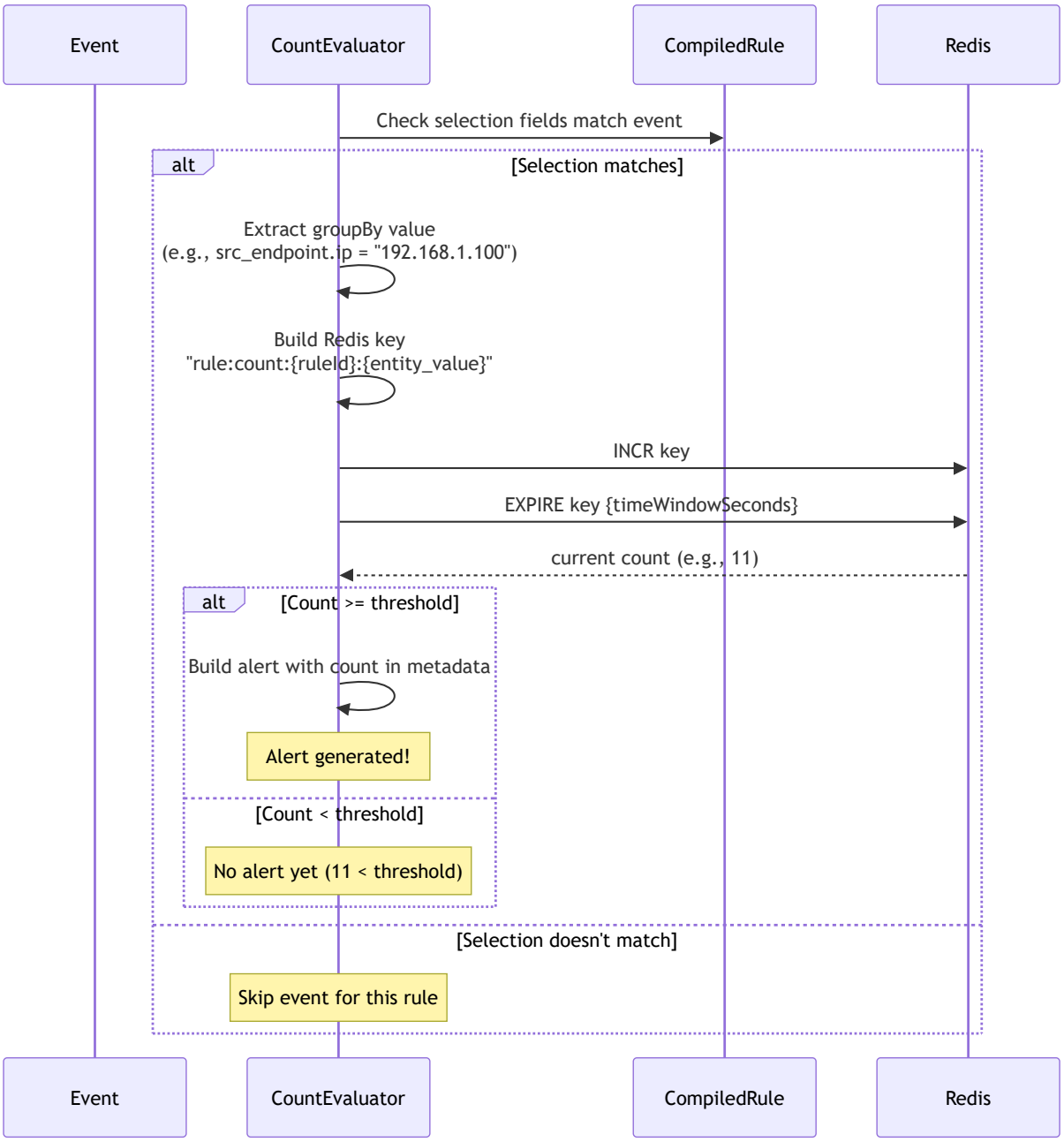


Example: Audit Log Cleared (Match Rule)

```yaml
title: Audit Log Cleared
id: rule-audit-clear-001
level: critical
weight: 0.95
priority: 900
logsource:
  class_uid: 6003    # System Activity
detection:
  selection:
    message|contains: "audit log cleared"
  selection_win:
    class_uid: 6003
    message|contains: "1102"
  condition: "selection OR selection_win"
tags:
  - attack.defense_evasion
  - attack.t1070
```

4.2 Count Rule Evaluation

Count rules track how many matching events occur within a rolling time window. An alert fires when the count reaches the threshold.



Redis State for Count Rules:

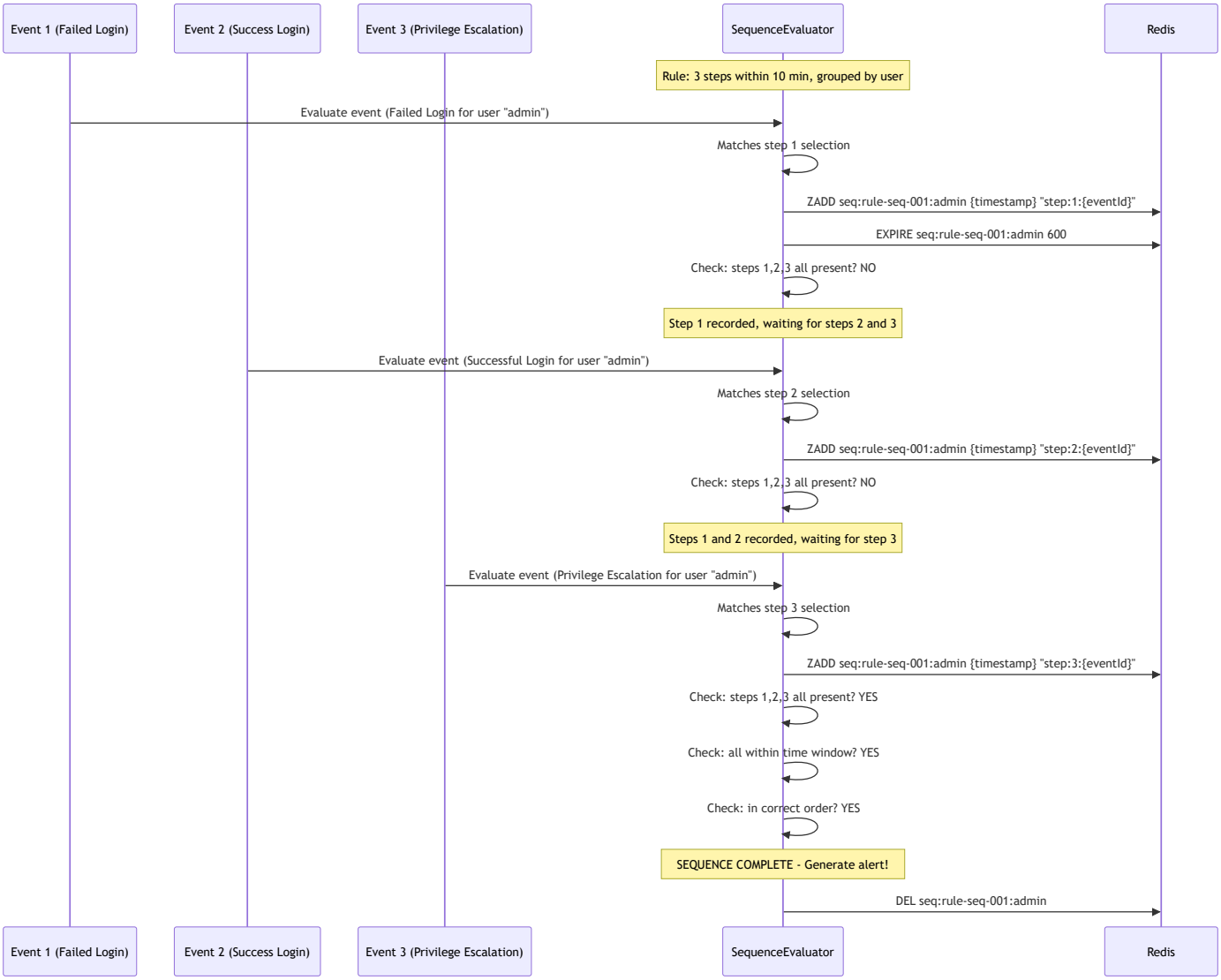
Redis Key	TTL	Value	Meaning
`rule:count:rule-bf-ssh-001:192.168.1.100`	300s (5m window)	`11`	11 failed logins from this IP in last 5 min
`rule:count:rule-bf-ssh-001:10.0.0.50`	300s	`3`	3 failed logins from this IP in last 5 min
`rule:count:rule-lockout-001:admin`	600s (10m)	`6`	6 lockout events for this user in last 10 min

Example: Brute Force Detection (Count Rule)

```yaml
title: SSH Brute Force
id: rule-bf-ssh-001
level: high
weight: 0.8
priority: 800
logsource:
  class_uid: 3002    # Authentication
detection:
  selection:
    message|contains: "Failed password"
    severity_id|gte: 3
  condition: "selection"
  count:
    field: "src_endpoint.ip"
    threshold: 10
    timewindow: "5m"
alert:
  cooldown: "5m"
tags:
  - attack.credential_access
  - attack.t1110
```

4.3 Sequence Rule Evaluation

Sequence rules detect ordered multi-event patterns. Events must occur in the defined order, from the same entity, within a time window.

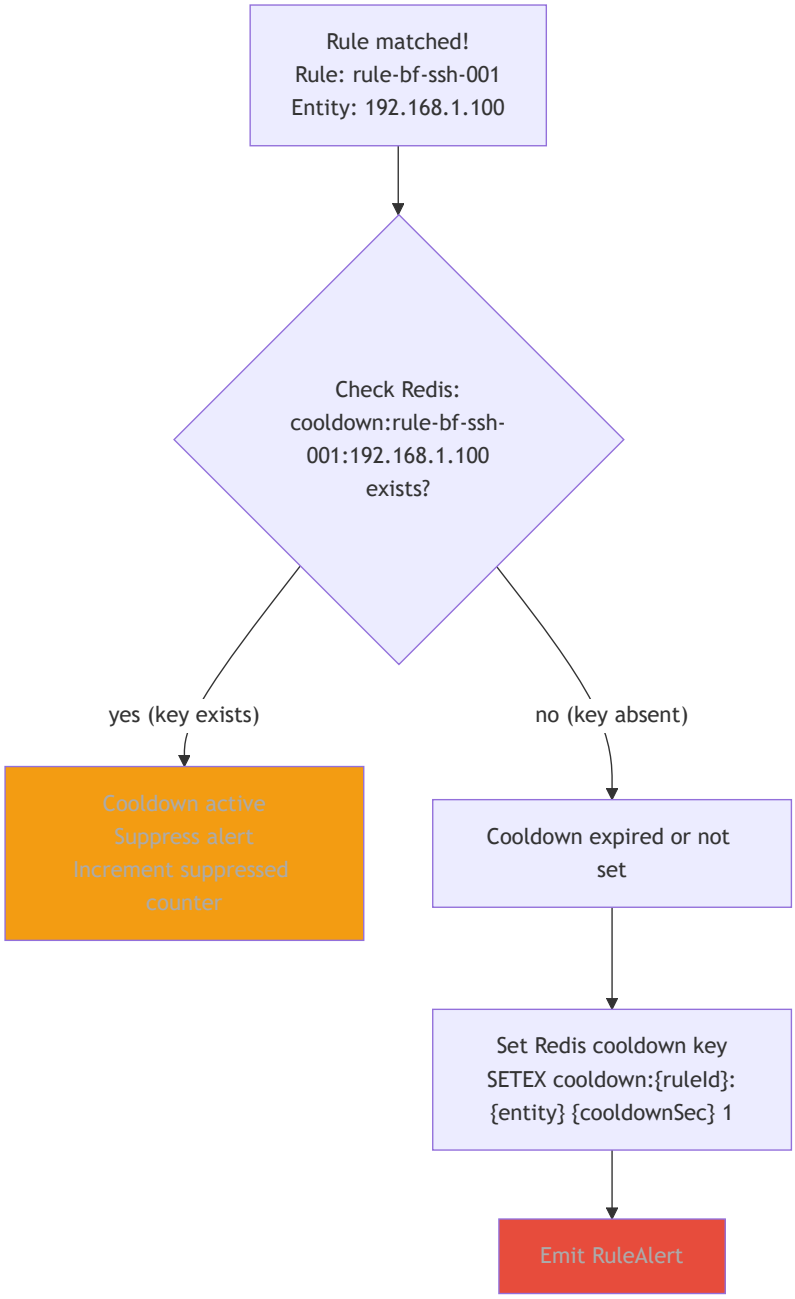


Example: Login Failure then Success with Privilege Escalation (Sequence Rule)

```yaml
title: Suspicious Login Sequence
id: rule-seq-login-001
level: high
weight: 0.85
priority: 850
logsource:
  class_uid: 3002
detection:
  condition: "sequence"
  sequence:
    timewindow: "10m"
    by: "actor.user.name"
    steps:
      - step1:
          message|contains: "Failed password"
          severity_id|gte: 3
      - step2:
          message|contains: "Accepted"
          severity_id: 1
      - step3:
          class_uid: 3004
          message|contains: "privilege"
alert:
  cooldown: "15m"
tags:
  - attack.initial_access
  - attack.privilege_escalation
```

4.4 Alert Cooldown

After a rule fires for a specific entity, it enters a cooldown period. During cooldown, the same rule+entity combination does not generate additional alerts — this prevents alert storms.

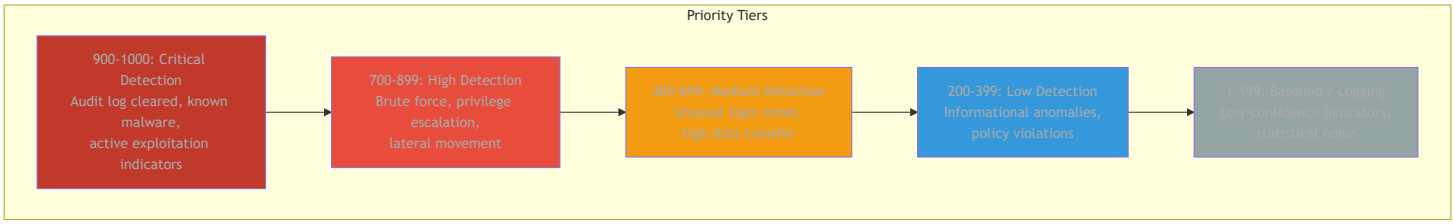


Redis Key	TTL	Purpose
`cooldown:rule-bf-ssh-001:192.168.1.100`	300s	Suppress brute force alerts for this IP for 5 min
`cooldown:rule-audit-clear-001:web-server-01`	60s	Suppress audit log cleared alerts for this host for 1 min

5. Rule Priorities

5.1 Priority System

Each rule has a `priority` field (1-1000). Higher numbers are evaluated first. This ensures critical rules fire before lower-priority rules and allows short-circuit optimizations.



5.2 Priority Resolution Rules

Scenario	Behavior
Same event, multiple rules match	All matching rules generate alerts. Priority determines evaluation order, not exclusivity
Same event, same priority	Rules at the same priority level are evaluated in alphabetical order by name (deterministic)
No priority specified	Default priority is `500` (medium)
Priority change via API	Triggers hot-reload. New priority takes effect on next evaluation cycle
Builtin vs custom rules	No priority difference. Custom rules can override builtin rule priority

5.3 Priority and Evaluation Order



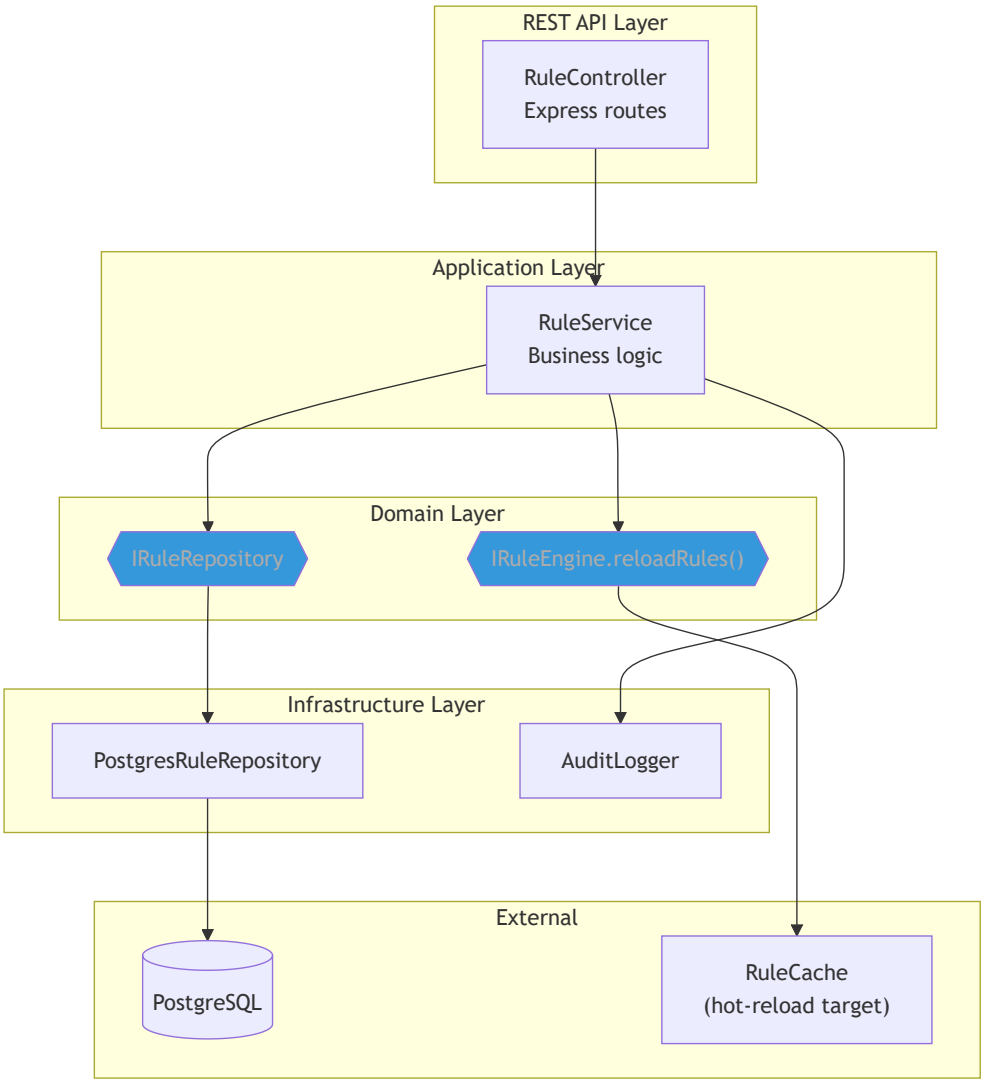
5.4 Priority and Performance

Priority ordering enables an optional **early termination** strategy for future optimization:

Strategy	Description	MVP Status
Evaluate all	Every applicable rule is evaluated regardless of matches	✅ MVP (default)
Max alerts per event	Stop after N rules match for a single event (configurable)	⏳ Post-MVP
Priority cutoff	Skip rules below a priority threshold under load	⏳ Post-MVP

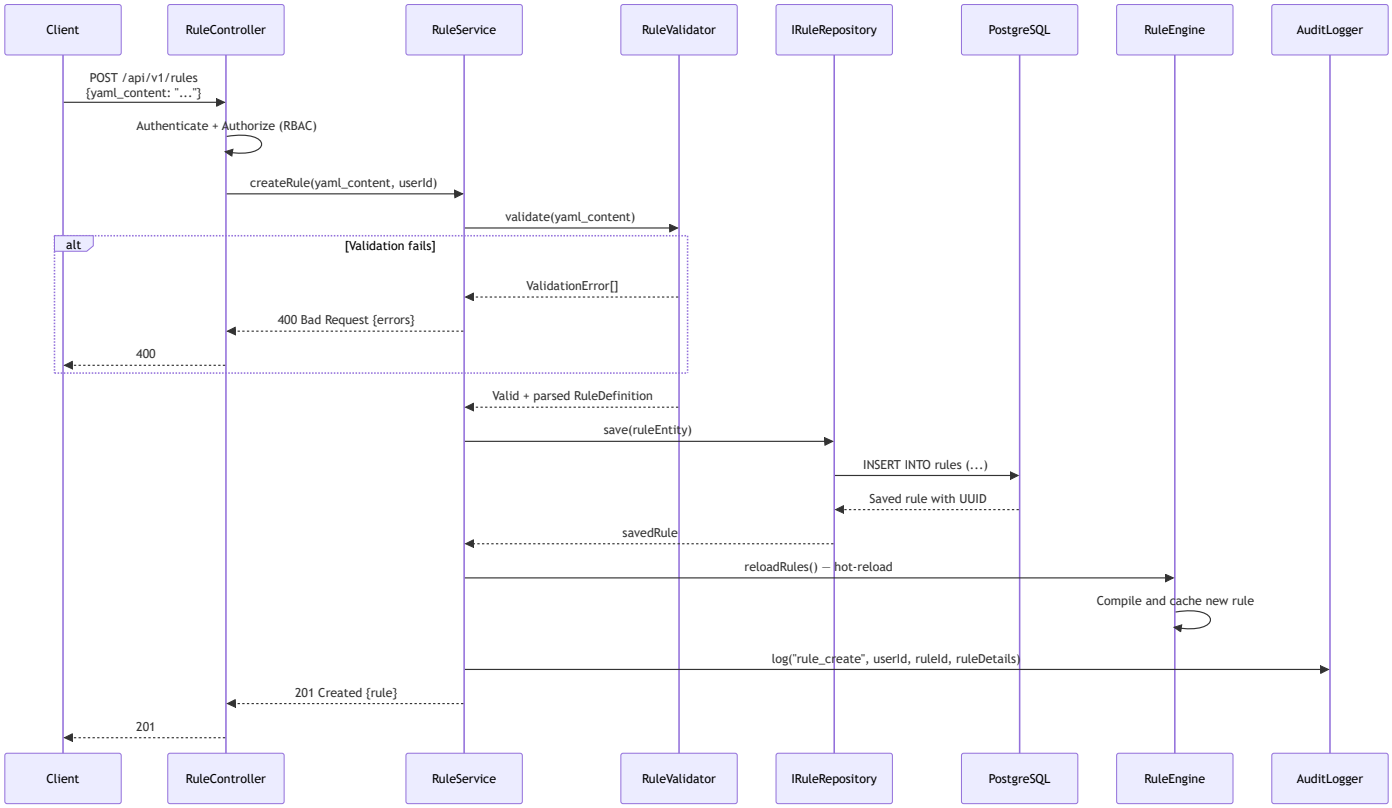
6. Rule CRUD

6.1 Rule CRUD Component Diagram

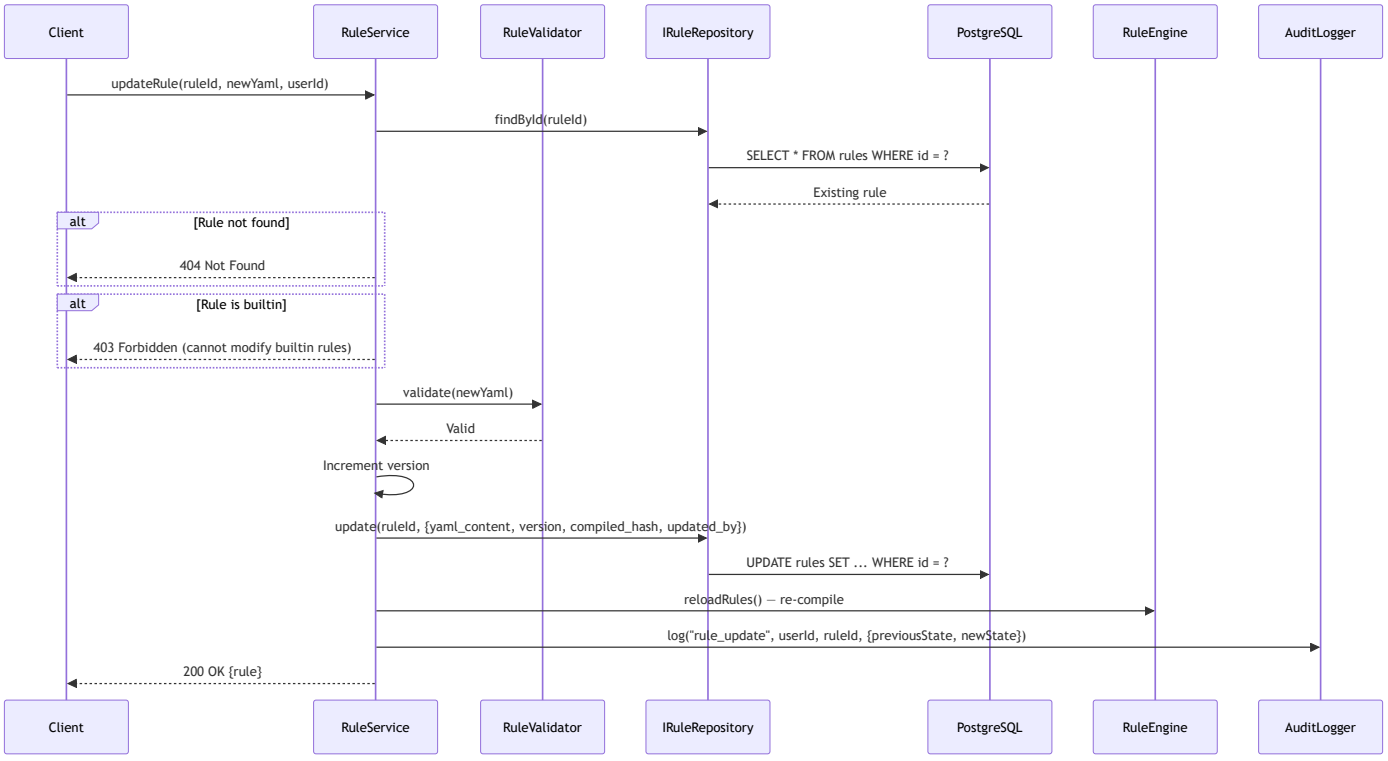


6.2 CRUD Operations

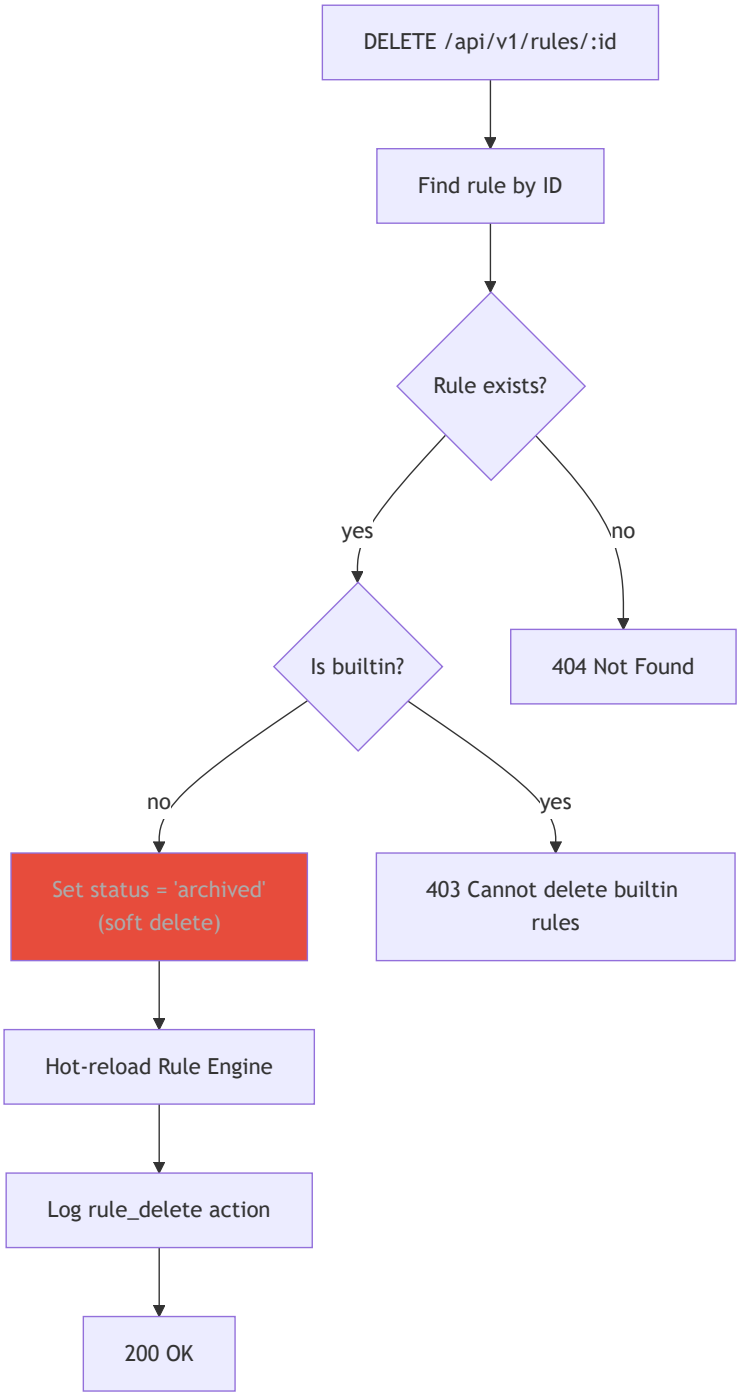
Create Rule



Update Rule



Delete Rule

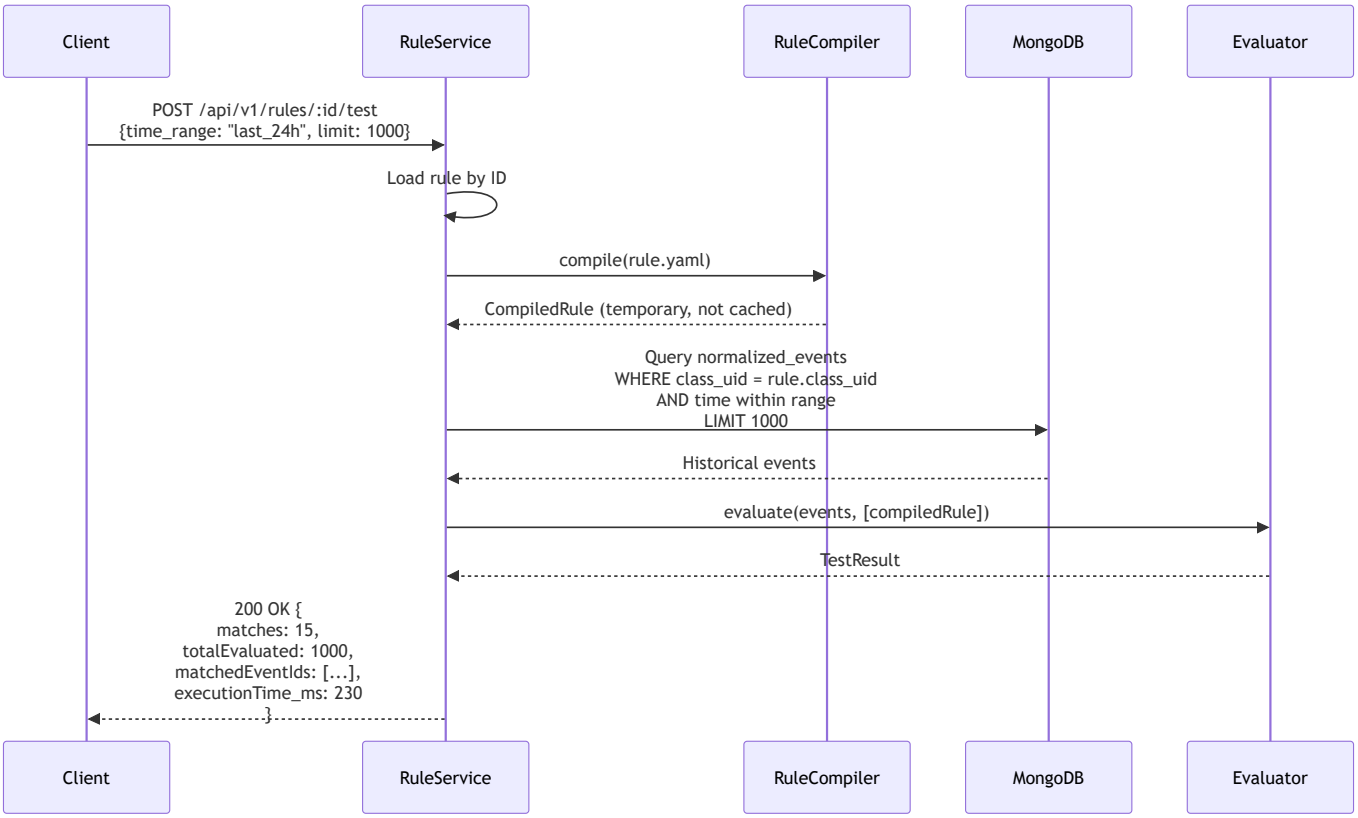


> **Important**
>
> Rules are **never hard-deleted**. Delete sets `status = 'archived'`. This preserves referential integrity with historical alerts and ensures the audit trail is complete.

Enable / Disable Rule

Operation	Endpoint	Effect
Enable	`PUT /api/v1/rules/:id/enable`	`status = 'active'`, hot-reload into engine
Disable	`PUT /api/v1/rules/:id/disable`	`status = 'disabled'`, hot-reload removes from engine

Test Rule (Dry-Run)



Import / Export Rules

Operation	Endpoint	Format	Description
Import	`POST /api/v1/rules/import`	YAML (multi-document: `---` separated)	Bulk import Sigma rules. Validates each, skips invalid
Export	`GET /api/v1/rules/export`	YAML archive (all active rules)	Download all rules as multi-document YAML

6.3 CRUD Validation Rules

Validation	Rule	Error
Title required	`title` must be non-empty string	`"Rule title is required"`
ID format	`id` must be alphanumeric + hyphens	`"Invalid rule ID format"`
ID unique	`id` must not exist in database (on create)	`"Rule ID already exists"`
Level valid	`level` must be a valid severity	`"Invalid severity level"`
Weight range	`weight` must be 0.0 - 1.0	`"Weight must be between 0.0 and 1.0"`
Priority range	`priority` must be 1-1000	`"Priority must be between 1 and 1000"`
Class UID valid	`logsource.class_uid` must be a known OCSF class	`"Unknown OCSF class UID"`
Condition parseable	`detection.condition` must be valid boolean expression	`"Cannot parse condition expression"`
Fields resolvable	All field paths in selections must be valid event paths	`"Unknown field: {fieldPath}"`
Count threshold	`count.threshold` must be positive integer	`"Count threshold must be > 0"`
Time window format	`count.timewindow` must match `\d+[smh]`	`"Invalid time window format"`
Sequence steps	`sequence.steps` must have >= 2 entries	`"Sequence rules need at least 2 steps"`

Validation	Rule	Error
YAML syntax	YAML must parse without errors	`"Invalid YAML syntax: {error}"`

6.4 Rule RBAC Permissions

Operation	Admin	Security Engineer	SOC Analyst
List rules	✅	✅	✅ (read-only)
View rule detail	✅	✅	✅ (read-only)
Create rule	✅	✅	❌
Update rule	✅	✅	❌
Delete rule	✅	❌	❌
Enable/Disable	✅	✅	❌
Import rules	✅	✅	❌
Export rules	✅	✅	✅
Test rule (dry-run)	✅	✅	✅

7. Performance Optimization

7.1 Optimization Architecture



7.2 Compile-Time Optimizations

Pre-Compiled Condition Functions

The most critical optimization. Instead of interpreting YAML at evaluation time, each rule's condition is compiled to a native JavaScript function:

```
YAML parse (once at load)
    ↓
Condition AST (tree structure)
    ↓
JavaScript function (called per event)
```

Approach	Cost per Event	1000 events × 50 rules
Interpret YAML at evaluation	~500µs	~25,000ms (25s)
Pre-compiled function call	~2µs	~100ms

Approach	Cost per Event	1000 events × 50 rules
Speedup		**~250x faster**

Class-Based Index

Events are dispatched only to rules matching their `class_uid`. With 50 rules across 10 OCSF classes, each event only evaluates ~5 rules instead of 50.

```
Without index:  50 rules × 1000 events = 50,000 evaluations
With index:      5 rules × 1000 events =  5,000 evaluations
Reduction:       90%
```

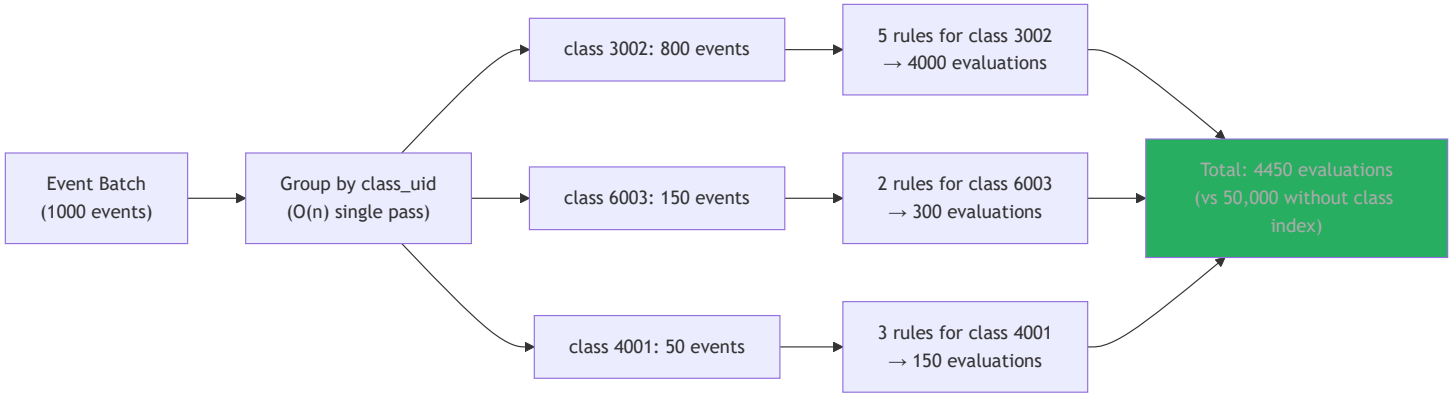
Compiled Regex Cache

Regex patterns from `|re` operators are compiled to `RegExp` objects once and stored in the `CompiledRule`. Per-event evaluation calls `.test()` on the pre-compiled regex.

7.3 Runtime Optimizations

Batch Evaluation Pipeline

Events are processed in batches (one batch per worker job, typically 1000 events). The Rule Engine evaluates the entire batch efficiently:

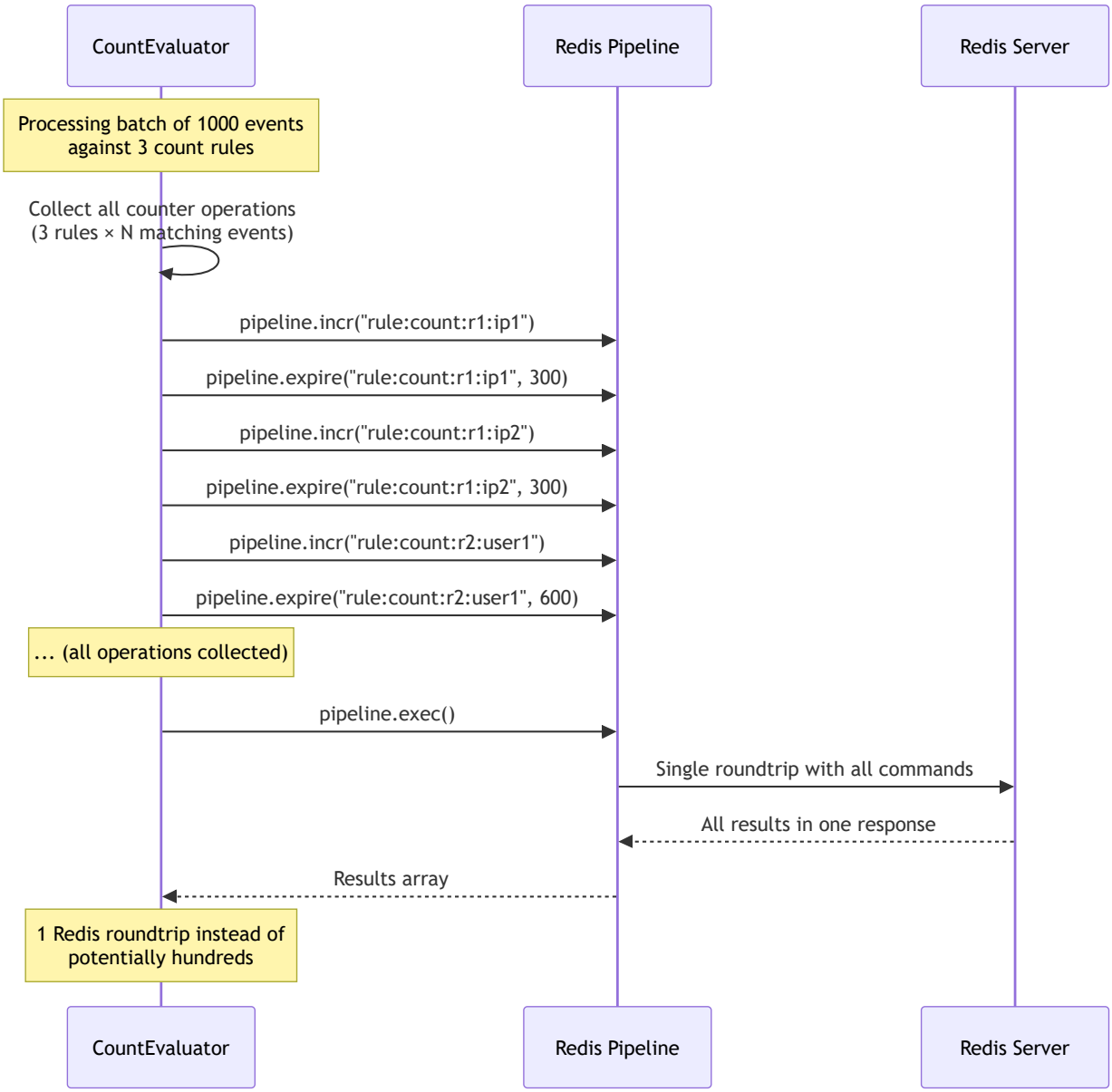


Lazy Field Resolution

Fields are resolved **only when the condition function accesses them**. If a condition checks `message|contains: "Failed"` first and that fails, deeper fields like `features.authentication.failed_login_count_10min` are never resolved.

7.4 Redis Pipeline Optimization

Count and Sequence rules require Redis operations. Instead of individual Redis calls per rule per event, operations are batched using Redis pipelining:



Approach	Redis Roundtrips (1000 events, 3 count rules)	Latency
Individual calls	Up to 6000 (2 per match: INCR + EXPIRE)	~300ms
Pipelined	1	~5ms

7.5 Performance Benchmarks (Target)

Metric	Target	Notes
Event evaluation throughput	10,000 events/sec per worker	With 4 workers: 40,000 EPS
Single match rule evaluation	< 5µs per event-rule pair	Pre-compiled function call
Single count rule evaluation	< 50µs per event-rule pair	Includes Redis pipeline contribution
Rule reload (hot-reload)	< 100ms for single rule	Delta compilation
Full reload (startup)	< 2s for 100 rules	Compile all + build indexes
Memory per compiled rule	~2KB	Function + metadata + index entries
Max active rules	500	Soft limit; beyond this, evaluate performance

Metric	Target	Notes
Cooldown check	< 10µs per check	Redis GET (pipelined)

7.6 Performance Monitoring

Metric	Collection	Alert Threshold
`rule_engine.evaluation_duration_ms`	Per batch	> 500ms (warning), > 2000ms (critical)
`rule_engine.rules_evaluated_per_second`	Rolling	< 1000 (warning — may be underloaded or stuck)
`rule_engine.alerts_per_minute`	Rolling	> 1000 (warning — possible alert storm)
`rule_engine.compile_errors`	Per reload	> 0 (warning — broken rule)
`rule_engine.cooldown_suppressions`	Rolling	Informational (track effectiveness)
`rule_engine.redis_pipeline_duration_ms`	Per batch	> 50ms (warning)
`rule_engine.cache_size`	Gauge	> 500 rules (warning)

7.7 Future Performance Enhancements

Enhancement	Description	Complexity	Impact
WASM-compiled conditions	Compile rule conditions to WebAssembly instead of JS functions	High	2-5x faster evaluation
Bloom filter pre-check	Bloom filter on common field values to skip events before evaluation	Medium	Reduce unnecessary evaluations by ~30%
Max alerts per event	Stop evaluating rules after N matches for a single event	Low	Reduce alert volume under load
Priority cutoff under load	Skip rules below priority threshold when queue depth exceeds limit	Low	Graceful degradation
Rule sharding	Distribute rules across workers by class_uid partition	Medium	Linear scaling with workers

> **This document is governed by [ADR-001](file:///d:/AI%20SIEM/docs/architecture/decisions/ADR-001-modular-monolith.md). The Rule Engine operates within the Node.js backend monolith as part of the Analysis Module. Rules are stored in PostgreSQL (see [DB-001](file:///d:/AI%20SIEM/docs/database.md)), stateful counters use Redis, and alerts feed into the Incident Correlator (see [BACKEND-002](file:///d:/AI%20SIEM/docs/backend-detection.md)).**
>
> **Document Version**: 1.0
> **Last Updated**: 2026-07-18